

TI-68K GAME DEVELOPMENT TOOLBOX

Roadmap de skills IA, de portage et de création de jeux natifs pour TI-89, TI-92 Plus et Voyage 200.

Vision : construire quatre compétences complémentaires sur un socle C portable, avec une version PC/SDL pour le développement et les tests, puis une compilation native Motorola 68000 pour calculatrice.

01. Portage open source : SDL → TI-68K

Concept. Récupérer un jeu open source, idéalement écrit en C avec SDL, et adapter son moteur et son rendu à la calculatrice.

Exemples. Open Flappy Bird ; Open Prince of Persia ; petits jeux SDL sur GitHub.

Pipeline. Sources C + SDL → Analyse et séparation logique/rendu → Adaptation monochrome 160 × 100 → Compilation TI-68K

Livable. Un skill capable d'analyser un dépôt et d'accompagner son portage natif en conservant autant que possible la logique originale.

02. Reverse engineering : ROM Game Boy → C → TI-68K

Concept. Partir d'une ROM sans sources, analyser son fonctionnement avec Ghidra, puis reconstruire la logique du jeu en C avec l'aide de l'IA.

Exemples. Bubble Ghost ou d'autres jeux Game Boy simples.

Pipeline. ROM → Désassemblage Ghidra → Identification code/données/assets → Reconstruction C → Backend TI-68K

Livable. Un exécutable natif 68000, et non une ROM interprétée par un émulateur. Le processus reste semi-automatique et demande une validation du comportement.

03. Remasterisation : TI-BASIC → C

Concept. Comprendre un jeu TI-BASIC existant, réécrire son moteur en C autonome, puis le faire tourner sur PC et sur TI-68K.

Exemples. Anciens jeux TI-BASIC dont le code source est disponible.

Pipeline. Analyse du TI-BASIC → Réécriture en C standard sans bibliothèque dynamique → Version PC/SDL et tests → Backend TI-68K

Livable. Un skill de migration et de modernisation, avec un moteur unique et deux cibles de compilation.

04. Gameplay : pseudo-écran tactile au pavé numérique

Concept. Transformer les touches numériques en zones de l'écran et interpréter des séquences comme des gestes.

Exemples. Prototype inspiré de Fruit Ninja ; balayages et sélections de zones.

Pipeline. Mapping des touches 1-9 → Détection de séquences → Reconnaissance de gestes → Intégration au moteur de jeu

Livable. Une bibliothèque d'interactions réutilisable : 4 → 5 → 6 = balayage horizontal ; 7 → 5 → 3 = diagonale ; 2 → 5 → 8 = vertical.

05. Socle commun : Portable Game Runtime

Un noyau de jeu en C standard (états, règles, physique, collisions), indépendant des périphériques. Deux backends : SDL sur PC pour tester et déboguer ; TI-68K pour framebuffer LCD, clavier, timers et optimisations 68000.

game_init() | game_update() | game_render()
input_pressed() | draw_sprite() | draw_text() | draw_tilemap()

Roadmap proposée

Phase	Chantier	Résultat attendu
1	Runtime C / SDL / TI-68K	Socle commun fonctionnel
2	Open Flappy Bird	Premier portage SDL → TI
3	TI-BASIC → C	Premier remaster historique
4	Pavé numérique tactile	Prototype de gameplay original
5	Bubble Ghost	Premier reverse engineering ROM
6	Prince of Persia	Portage plus ambitieux

Organisation du dépôt

ti68k-toolbox/
 skills/ sdl-to-ti68k/ rom-to-c/ tibasic-to-c/ keypad-touch/
 runtime/ core/ platform-sdl/ platform-ti68k/
 tools/ asset-converter/ rom-analyzer/
 games/ flappy-bird/ bubble-ghost/ fruit-ninja/
 tests/

Principe directeur : un seul moteur C portable, un backend SDL pour itérer rapidement sur PC et un backend 68000 pour livrer sur calculatrice. Chaque skill dispose d'instructions, d'exemples, de scripts et de critères de validation.